Artificial Intelligence (AI-2002)

Chapter 5: Constraint Satisfaction Problem

Objectives

- Backtracking
- MRV and degree heuristic
- Forward checking
- Improvement in forward check (through heuristic combination)
- MAC (maintaining arc consistency)
- Back jumping
- Intelligent backtracking (constraint learning)

Constraint Satisfaction Problem

Standard search problem:

state is a "black box"—any old data structure
that supports goal test, eval, successor

CSP:

state is defined by variables X_i with values from domain D_i

goal test is a set of constraints specifying
allowable combinations of values for subsets of variables

Simple example of a **formal representation language**

Allows useful **general-purpose** algorithms with more power than standard search algorithms

Constraint Satisfaction Problem

- So far, Search space states evaluated by heuristics and goal.

- A constraint satisfaction problem (CSP) is a problem that requires its solution within some limitations/conditions also known as constraints.

- For which the typical problems can be converted to CSP and solved.

Converting problems to CSPs

- A problem to be converted to CSP requires the following steps:
- **Step 1:** Create a variable set.(Representing state)
- **Step 2:** Create a domain set.(values)
- **Step 3:** Create a constraint set with variables and domains after considering the constraints.(limitations)

CSP Example1-Sudoku

Rule: Fill all empty Squares so that the numbers 1 to 9 appear once in each row, column and 3x3 box

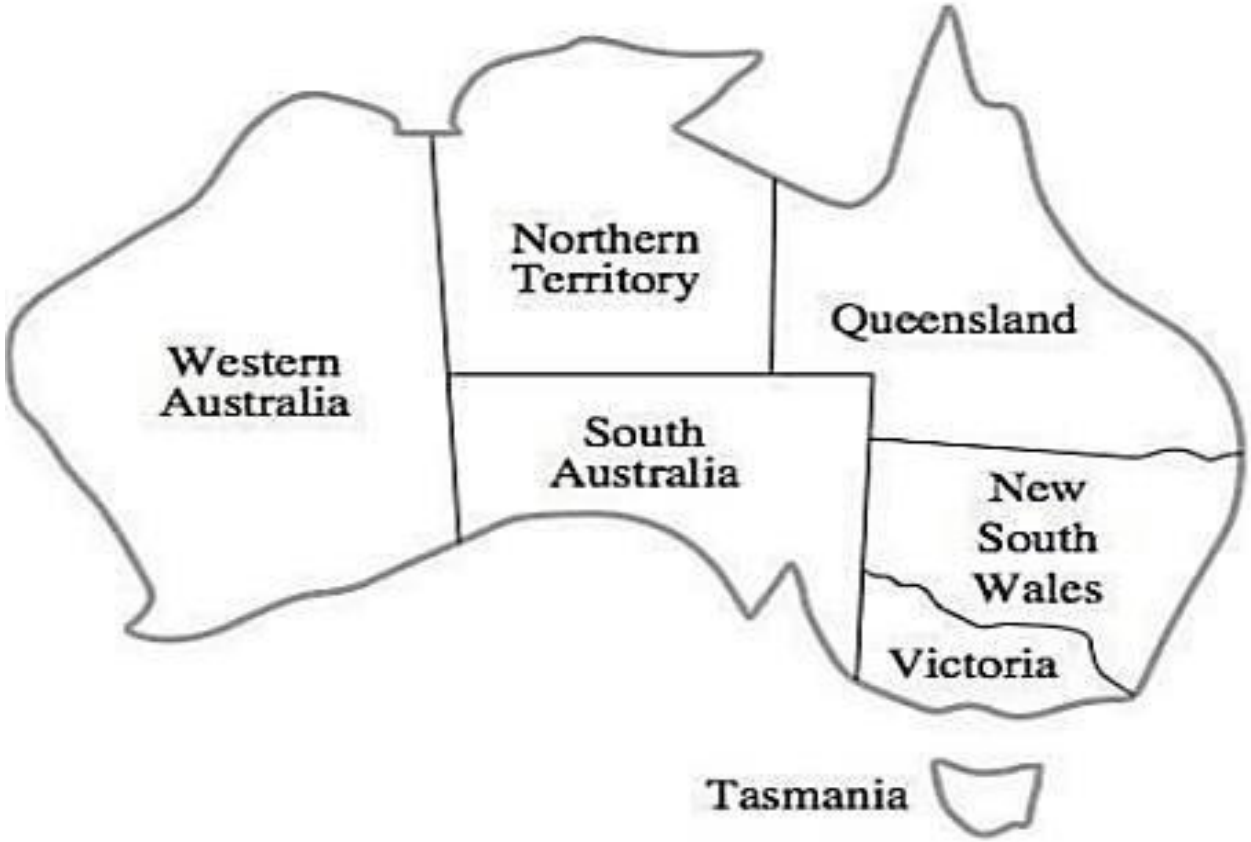
	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

VARIABLES – variables are cells (A1-I9)

DOMAIN – domain of each variable is {1,2,3,4,5,6,7,8,9}

CONSTRAINT– constraints: rows, columns, boxes contain all different numbers

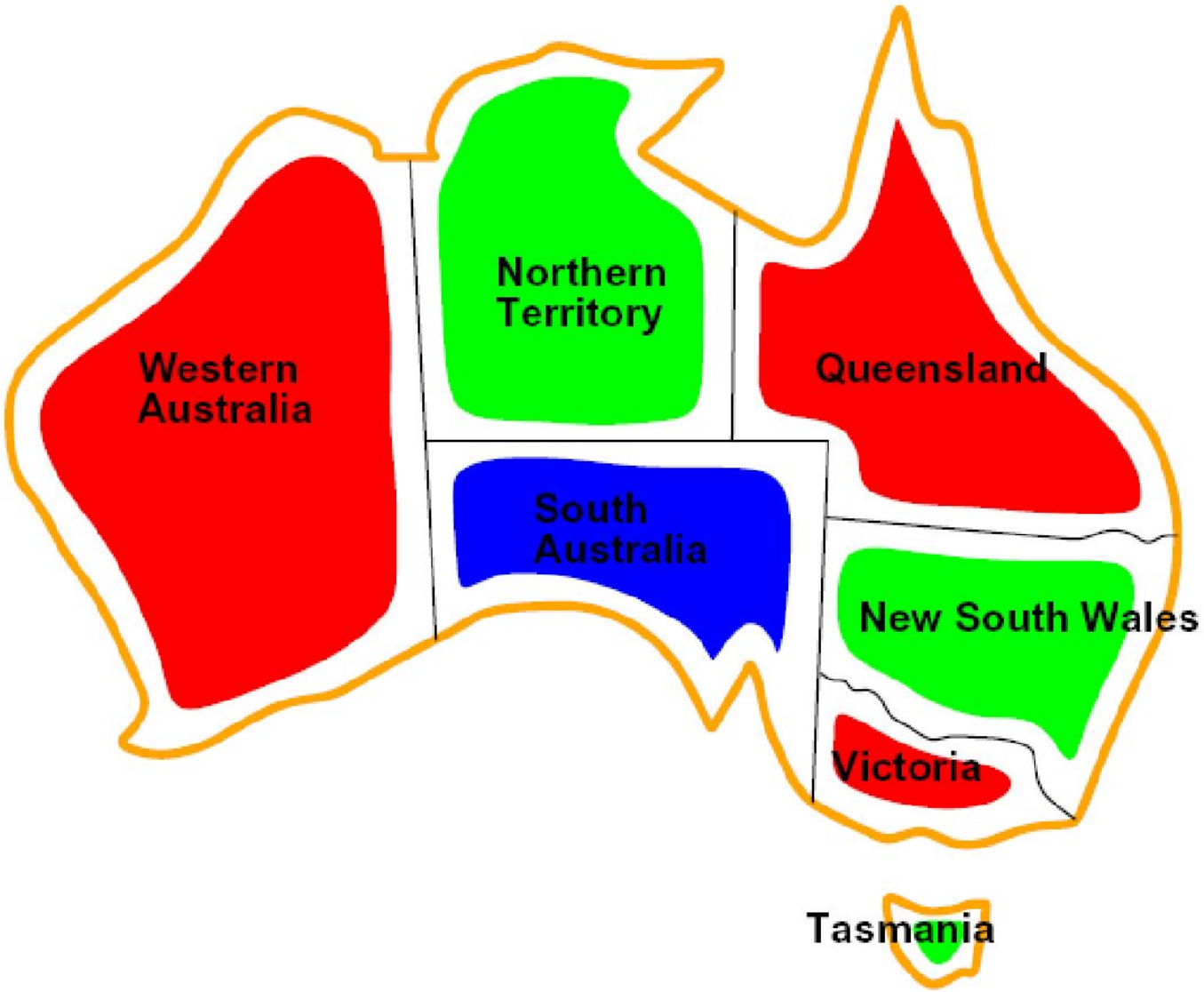
Example2-Map Coloring



Rule - Color each region either red, green or blue

- No adjacent region can have the same color

Map Coloring



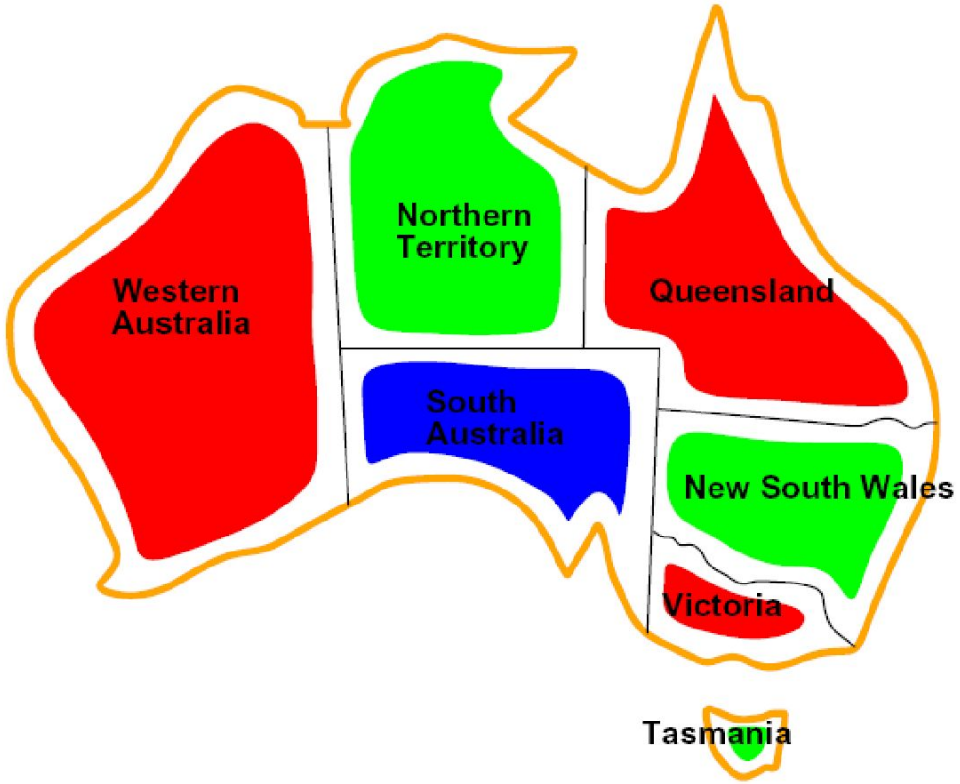
Map Coloring

Variables:
WA, NT, SA, Q, NSW, V, T

Domains:
D = {**R** , **G**, **B**}

Constraints:
adjacent regions must have different colors

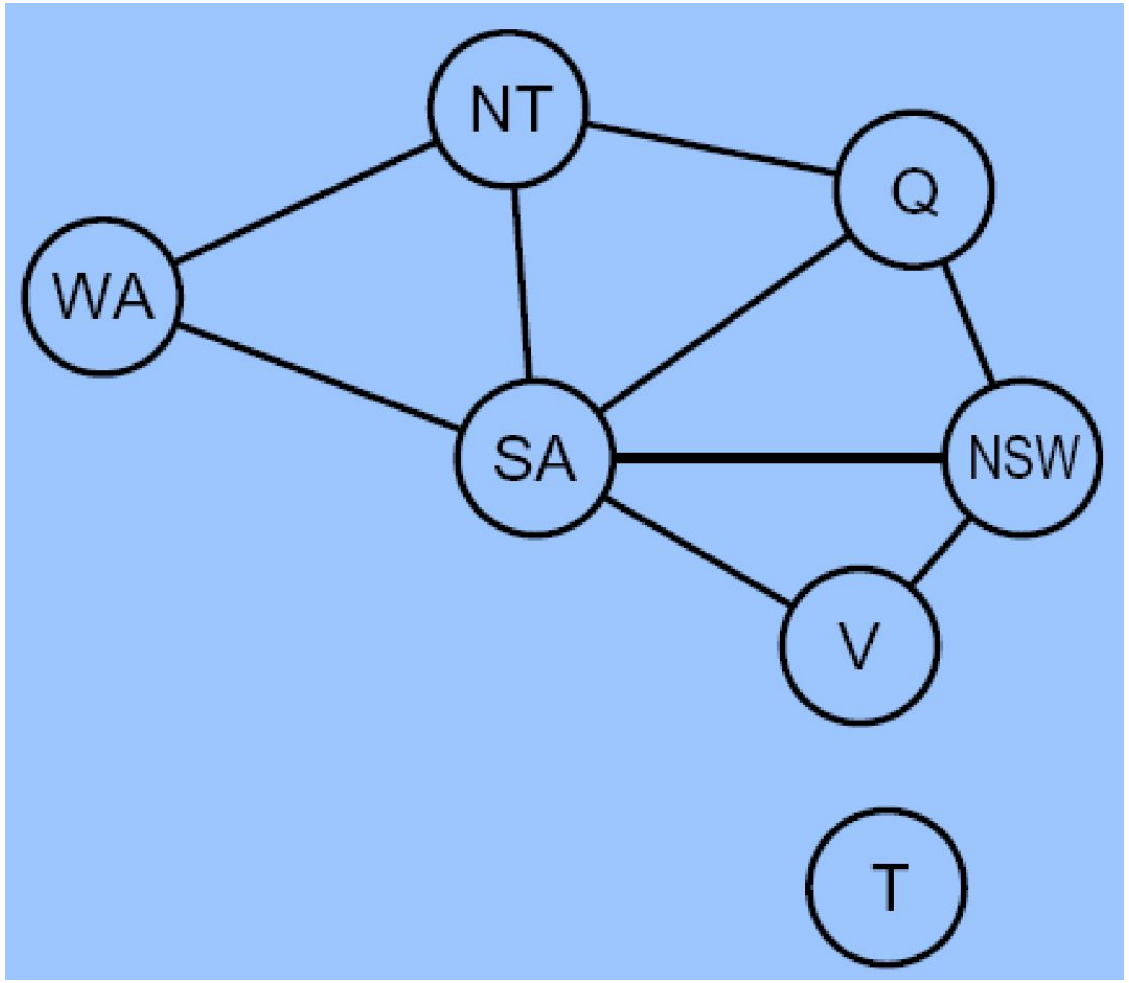
WA != NT
(WA, NT) in {(**R**,**G**), (**R**,**B**), (**B**,**G**), … }



Solutions are assignments satisfying all constraints, e.g.:

Constraint Graphs

	WA	NT	SA	Q	NSW	V
WA		1	1	0	0	0
NT	1		1	1	0	0
SA	1	1		1	1	1
Q	0	1	1		1	0
NSW	0	0	1	1	0	1
V	0	0	1	0	1	0

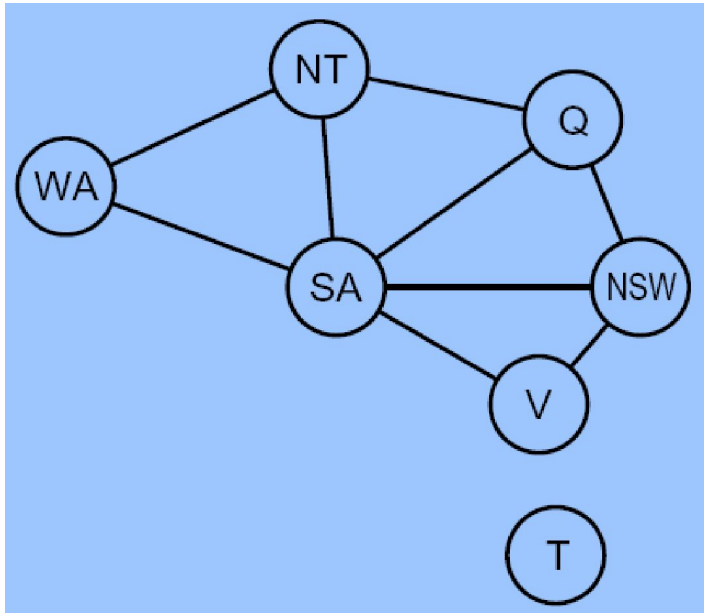


Constraint Graphs

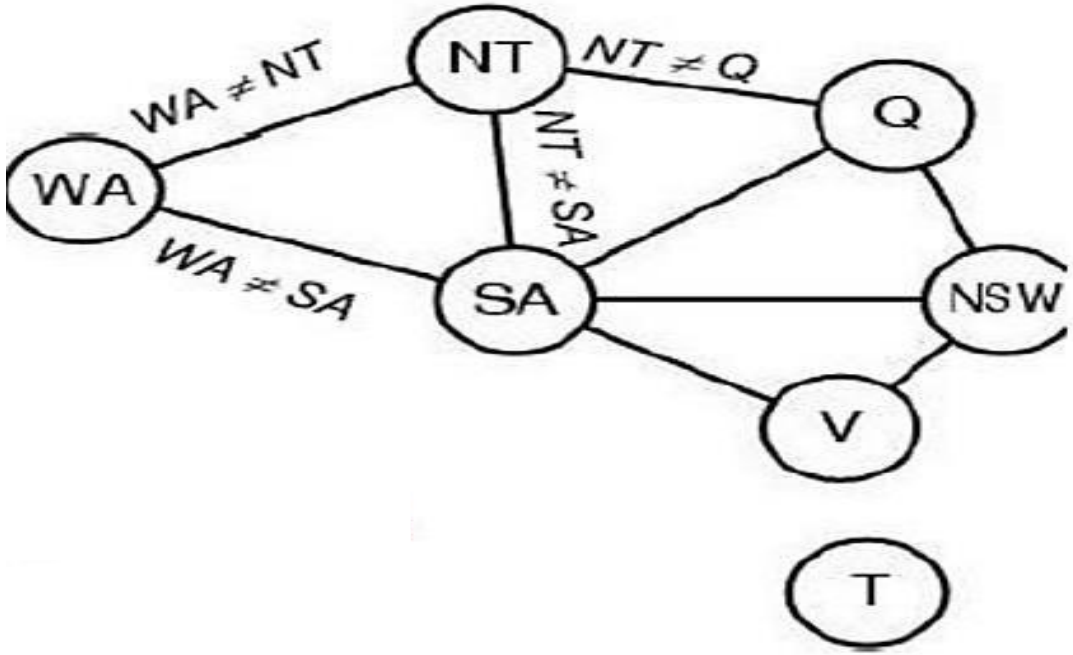
Binary CSP: each constraint relates (at most) two variables

Binary constraint graph: nodes are variables, arcs show constraints

General-purpose CSP algorithms use the graph structure to speed up search. E.g., Tasmania is an independent sub problem!



CSP Representation of Map Coloring



Variables: WA, NT, Q, NSW, V, SA, T

Domains: Di = {red, green, blue}

Constraints: adjacent regions must have different colors

e.g., WA ≠ NT, or (WA,NT)

Constraint Graph

- **Nodes** are Variables
- **Arcs** are Constraint

Varieties of constraints

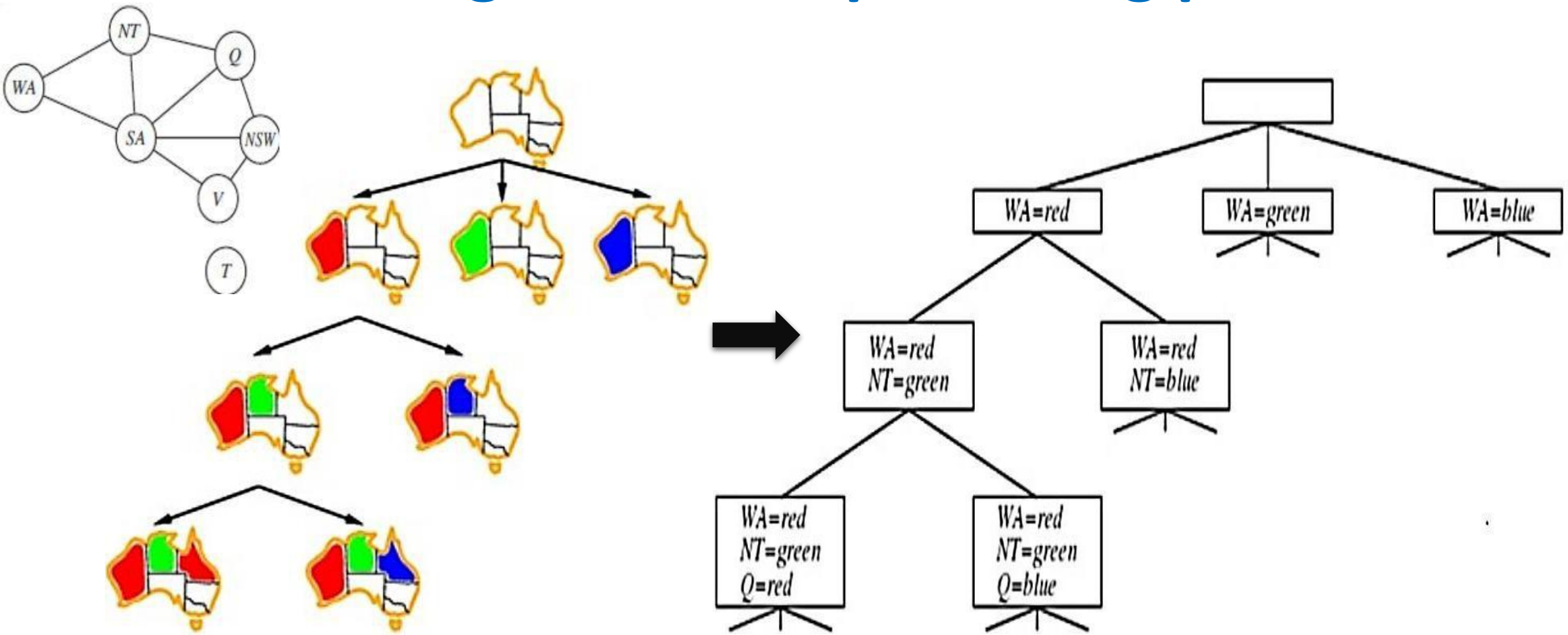
- Unary constraints involve a single variable, e.g. SA ≠ green
- Binary constraints involve pairs of variables (relates two variables),e.g., SA ≠ WA
- Higher-order constraints involve 3 or more variables, e.g. cryptarithmetic puzzles (column constraints)

BACKTRACKING

Backtracking search

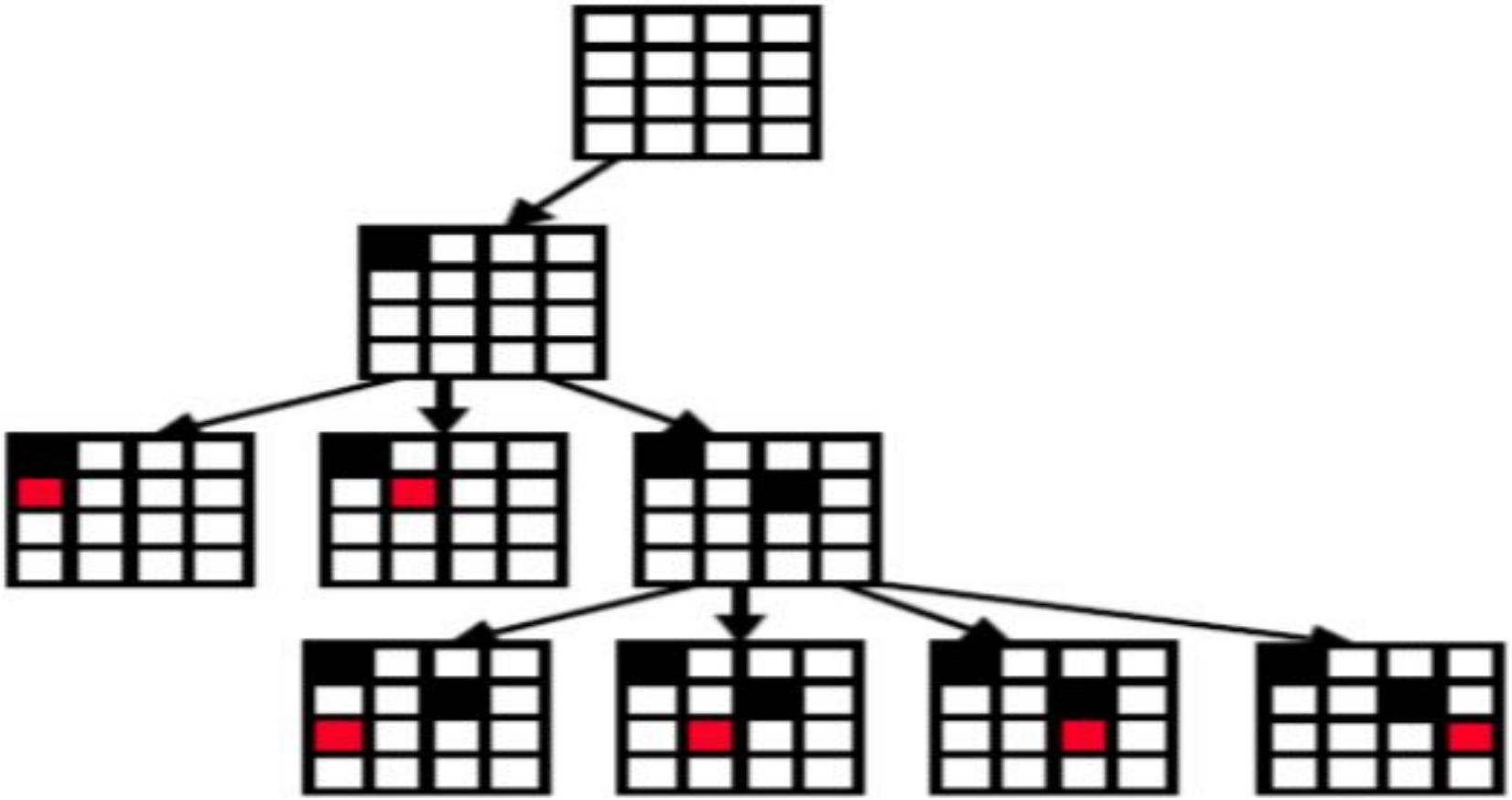
- Depth-first search for CSPs with single-variable assignments is called backtracking search
- Variable assignments are commutative: [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each node
- Backtracks when variable has no legal value

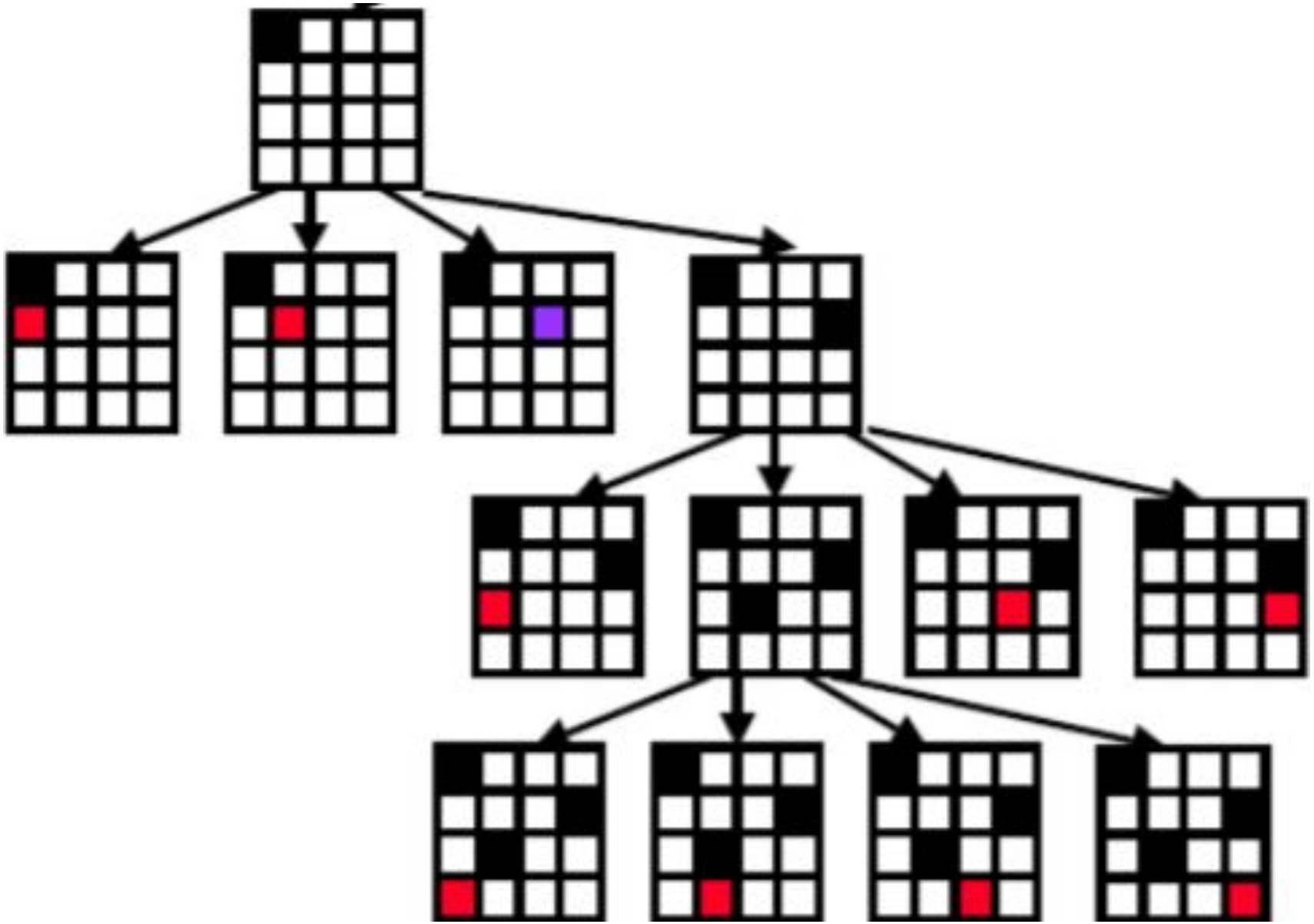
Backtracking for the map-coloring problem



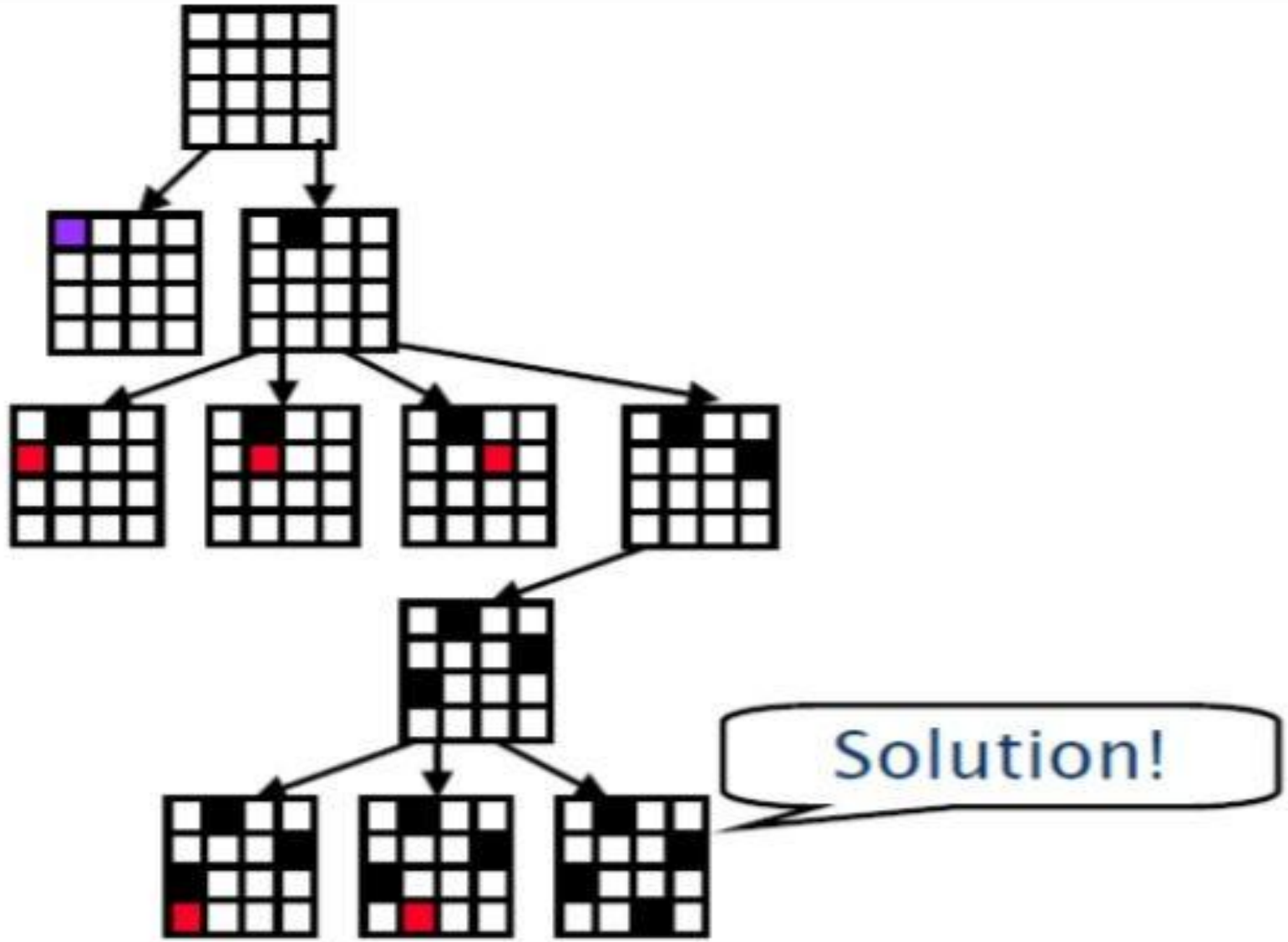
Backtracking Example:2

- 4X4 Queens





- 4X4 Queen



Backtracking search

Backtracking = DFS + **variable-ordering** + **fail-on-violation**

Improving backtracking efficiency (Variable and Value Ordering)

By default, Backtrack algorithm simply selects the next unassigned variable in the order given by the list VARIABLES[csp].

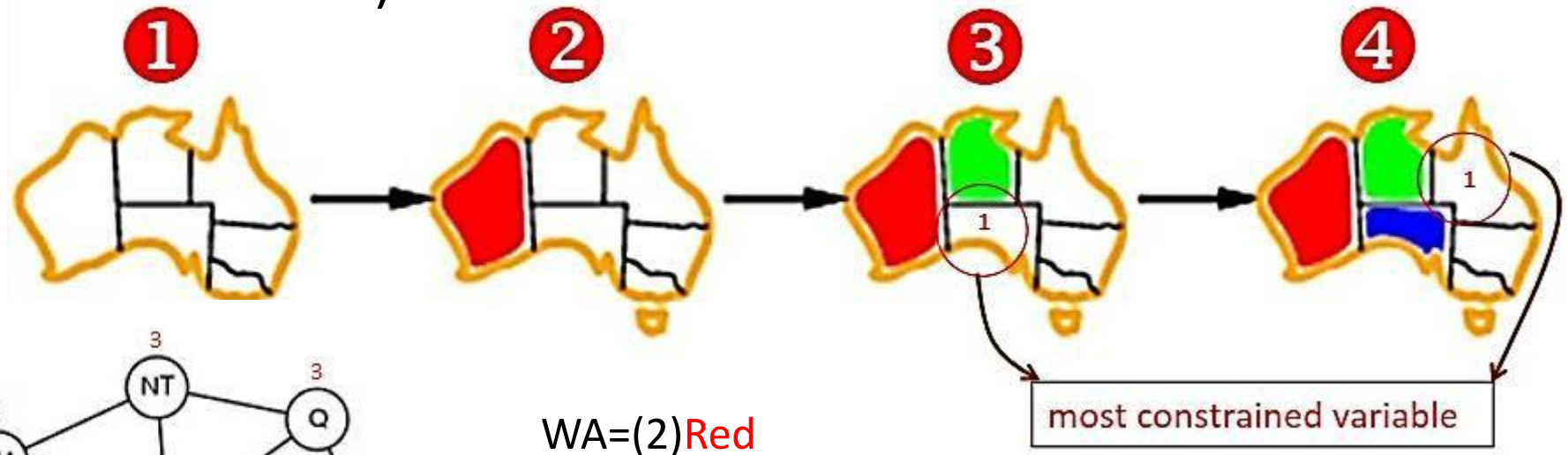
General-purpose methods can give huge gains in speed:

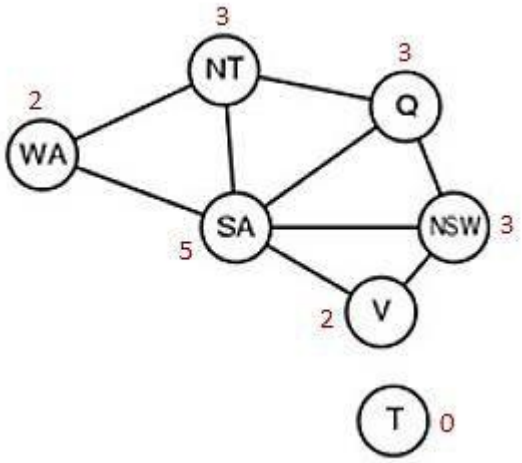
- Which variable should be assigned next?
- In what order should its values be tried ?
- Can we detect inevitable failure early?

Ordering

Minimum remaining values (MRV)

- This intuitive idea-choosing the variable with the fewest "legal" values is called the minimum remaining values (MRV) heuristic (also known most constrained variable)



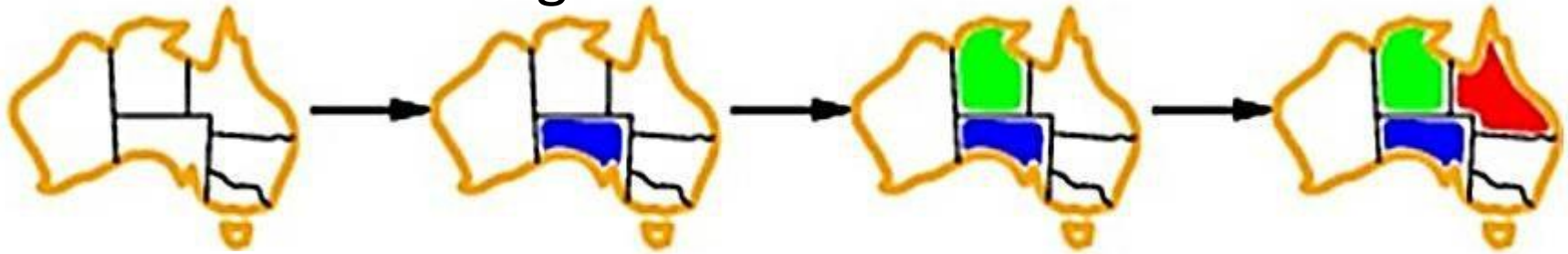


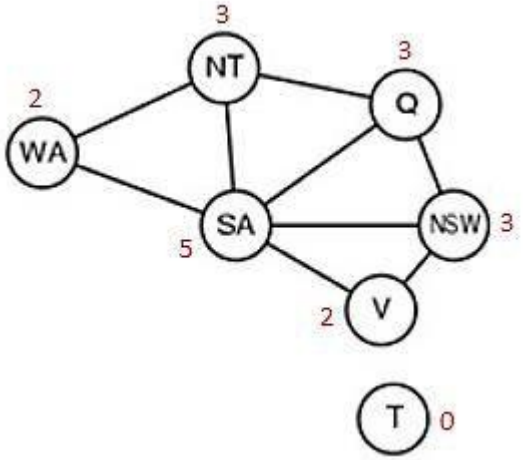
WA=(2)Red
NT(Remaining values)=2(Blue , Green)
while Q and NSW have 3 remaining values so, go for NT
SA(Remaining values)= 1(Blue)

after the assignments for WA = red and NT = green, there is only one possible value for SA, so it makes sense to assign **SA** = blue next rather than assigning Q, after SA is assigned, the choices for Q, NS W, and V are all forced.

Most-Constraining Variable (Degree Heuristics)

- Most constraining variable: choose the variable with the most constraints on remaining variables





SA=5 (Most Constraints) BLUE
NT, Q, NSW = 3 **TIE** b/w them break it by choosing the first choice either NT or NSW.
So, choose NT Then Q, NSW OR Q, Then Q and NT

Filtering

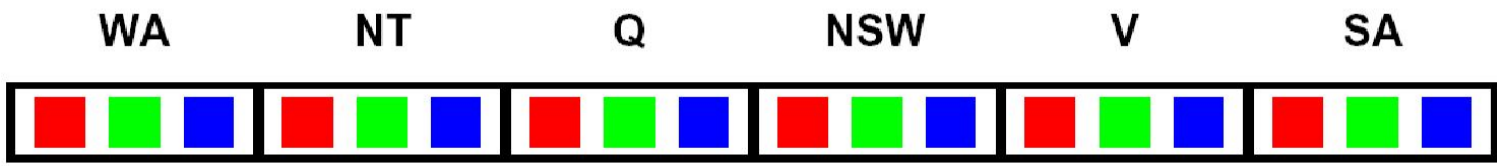
Filtering: Forward Checking

Filtering: Keep track of domains for unassigned variables and cross off bad options

Forward checking: Cross off values that violate a constraint when added to the existing assignment

Filtering: Forward Checking

Filtering: Keep track of domains for unassigned variables and cross off bad options
Forward checking: Cross off values that violate a constraint when added to the existing assignment



Forward Checking

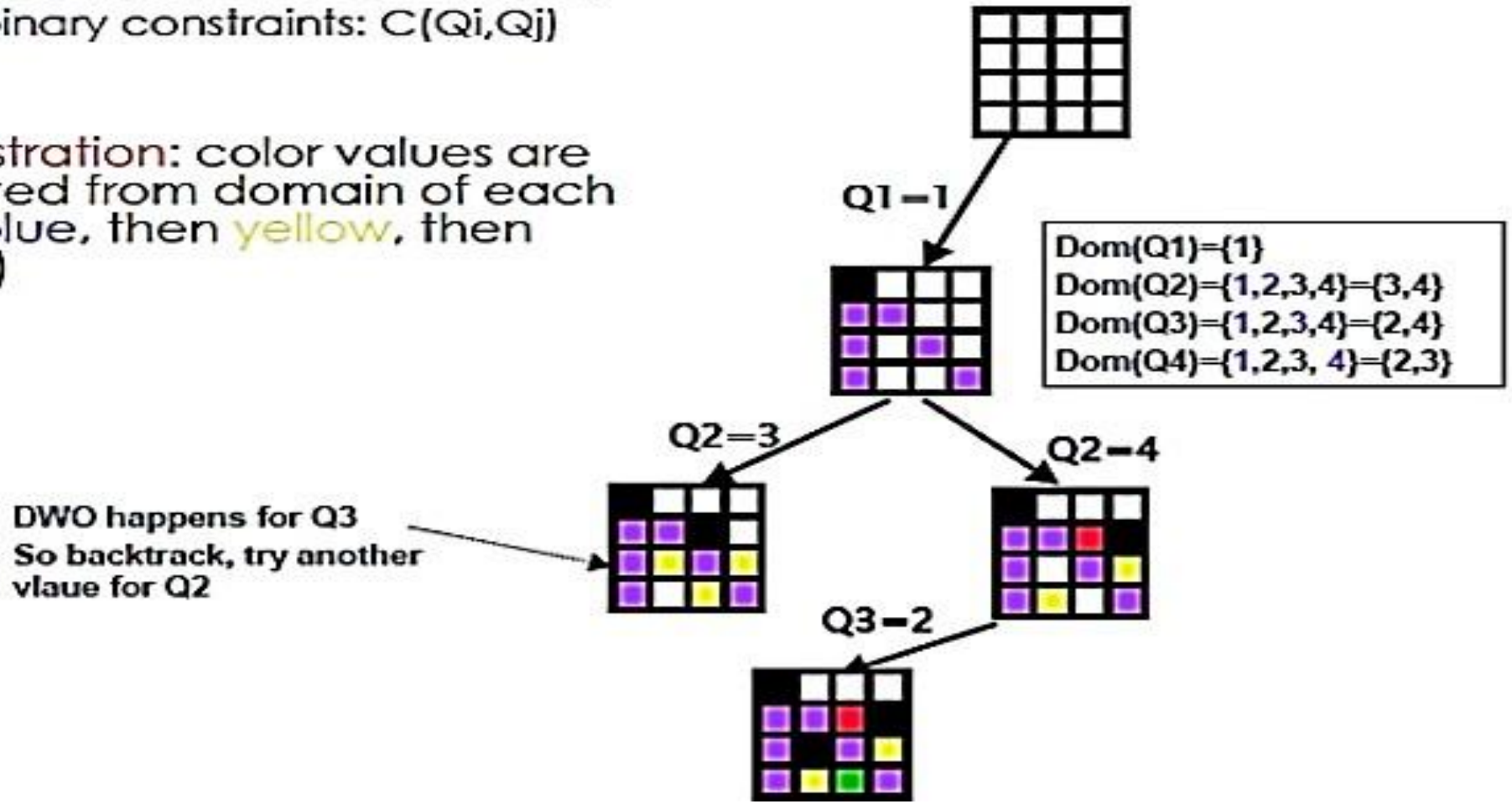
Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values

No possible assignments for SA, we try other assignments

Forward Checking Example:2

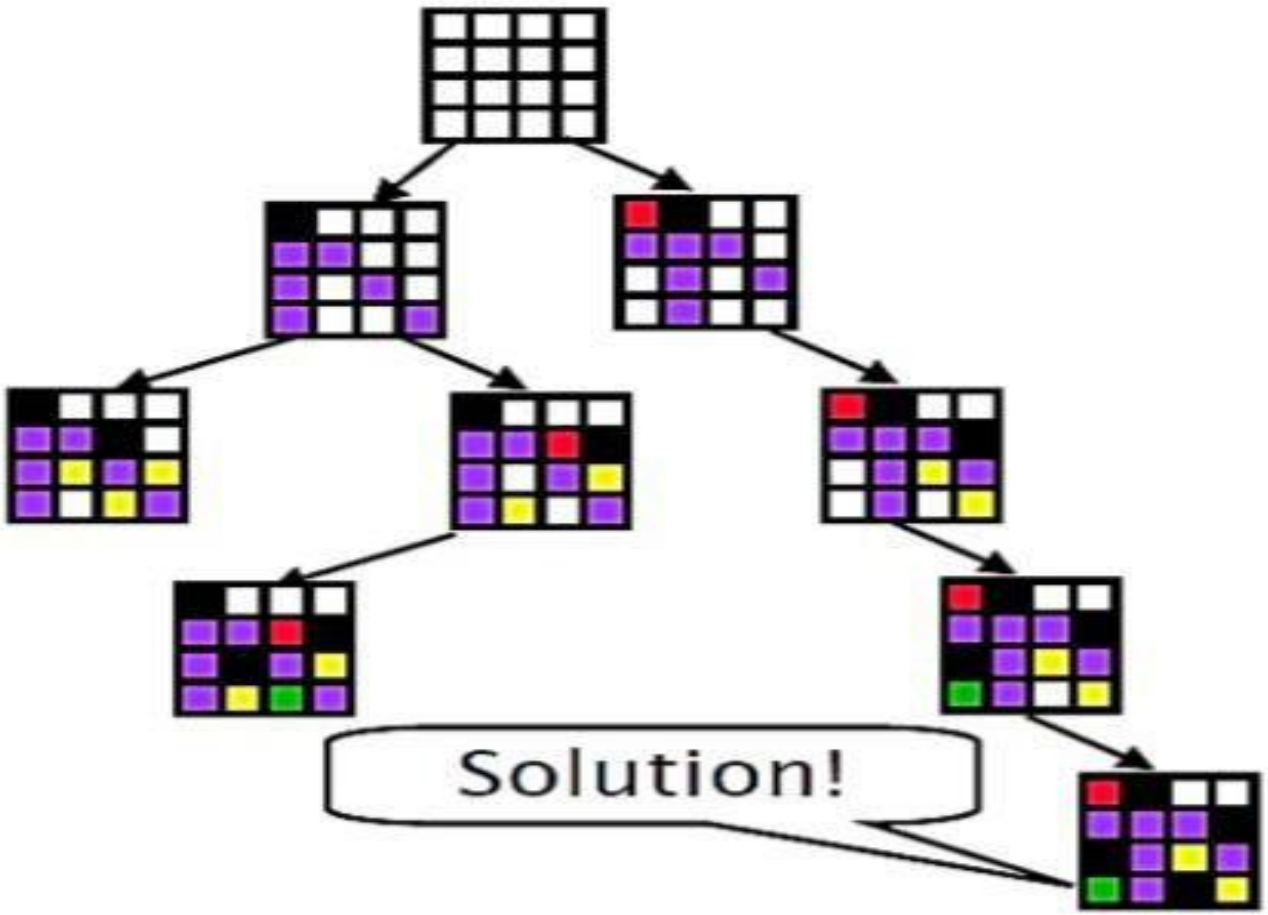
- 4X4 Queens
  - Q1,Q2,Q3,Q4 with domain {1..4}
  - All binary constraints: C(Qi,Qj)

- FC illustration: color values are removed from domain of each row (blue, then yellow, then green)



Forward Checking Example:2……

- 4X4 Queens continue…

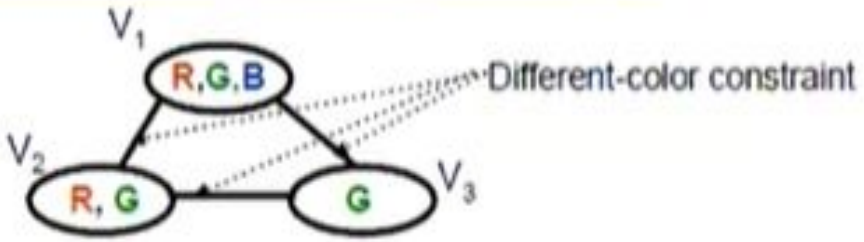


ARC Consistency

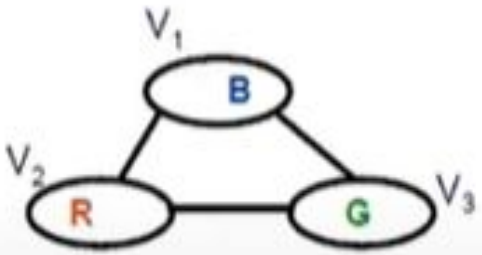
Constraint Propagation Example

Graph Coloring

Initial Domains are indicated

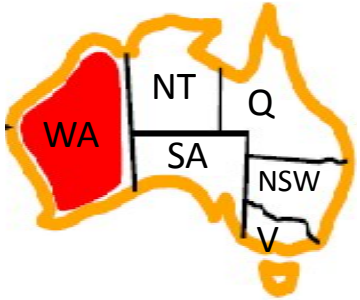


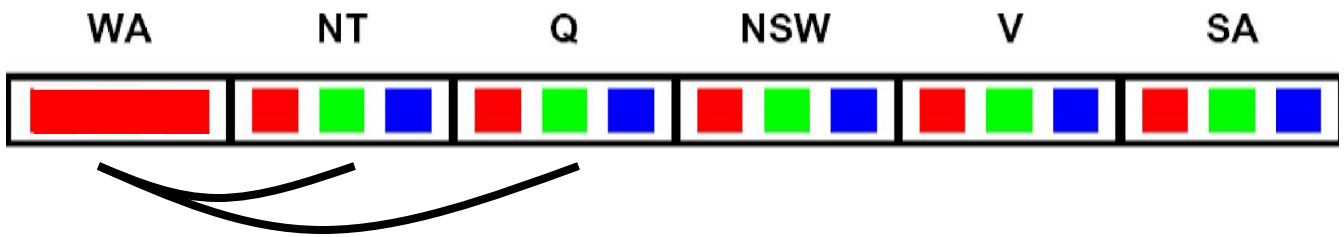
Arc examined	Value deleted
$V_1 - V_2$	none
$V_1 - V_3$	V_1(G)
$V_2 - V_3$	V_2(G)
$V_1 - V_2$	V_1(R)
$V_1 - V_3$	none
$V_2 - V_3$	none



Consistency of A Single Arc

An arc X ⟶ Y is consistent iff for every x in the tail there is some y in the head which could be assigned without violating a constraint



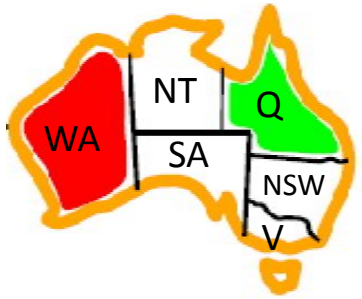


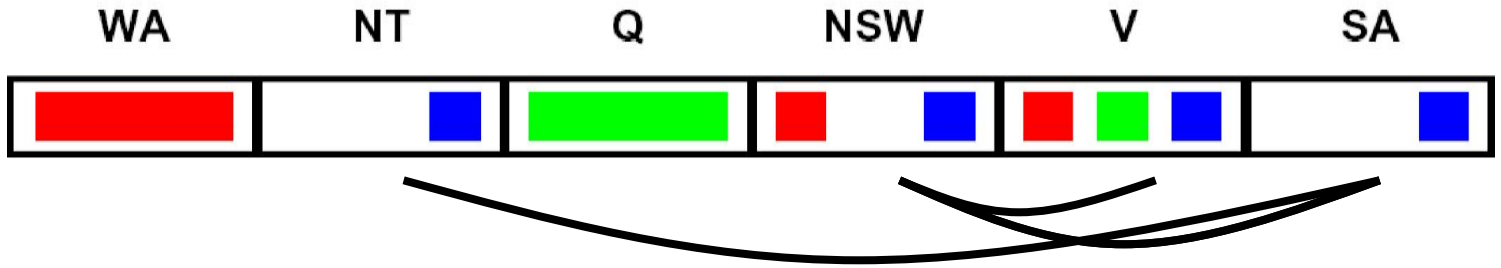
Forward checking: Enforcing consistency of arcs pointing to each new assignment

Delete from the tail!

Arc Consistency of an Entire CSP

A simple form of propagation makes sure all arcs are consistent:



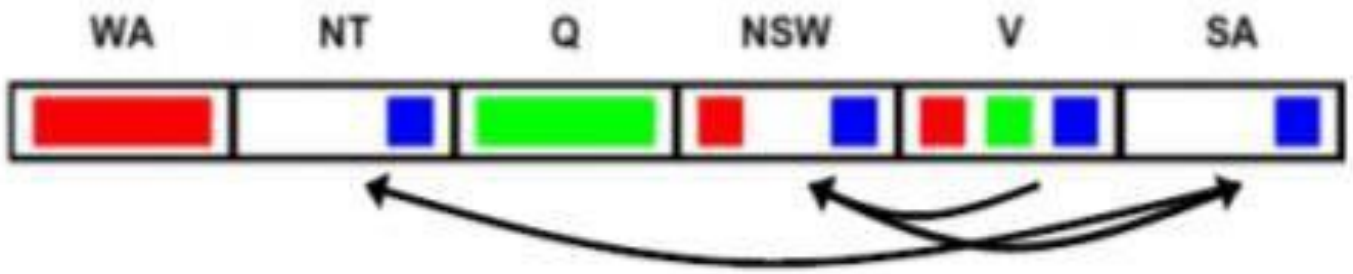


Important: If X loses a value, neighbors of X need to be rechecked!
Arc consistency detects failure earlier than forward checking
Can be run as a preprocessor or after each assignment
What's the downside of enforcing arc consistency?

Remember: Delete from the tail!

ARC Consistency

- A simple form of propagation makes sure all arcs are simultaneously consistent:



- Arc consistency detects failure earlier than forward checking
- Important: If X loses a value, neighbors of X need to be rechecked!
- Must rerun after each assignment!

Remember: Delete from the tail!

```
function AC-3( csp) returns the CSP, possibly with reduced domains
   inputs: csp, a binary CSP with variables {X1, X2, ..., Xn}
   local variables: queue, a queue of arcs, initially all the arcs in csp

   while queue is not empty do
      (Xi, Xj) ← REMOVE-FIRST(queue)
      if REMOVE-INCONSISTENT-VALUES(Xi, Xj) then
         for each Xk in NEIGHBORS[Xi] do
            add (Xk, Xi) to queue
```

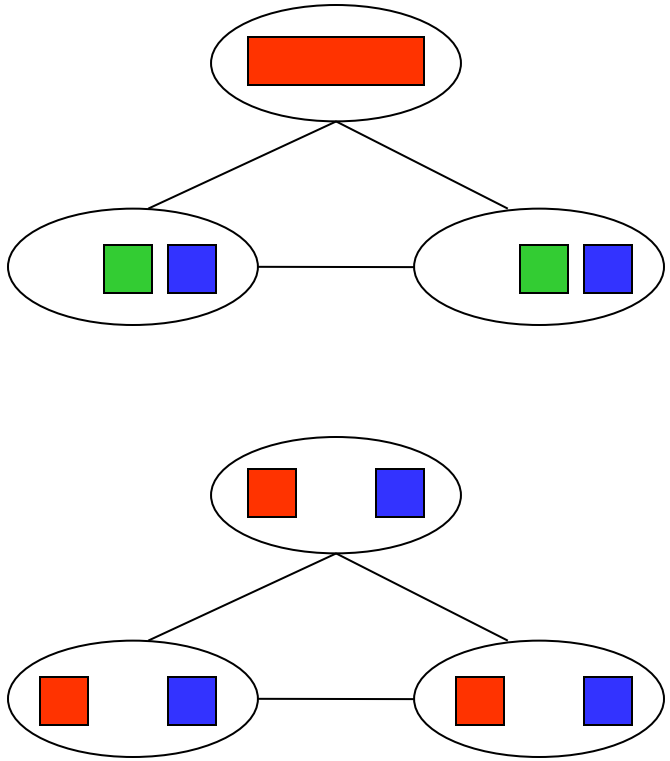
```
function REMOVE-INCONSISTENT-VALUES( Xi, Xj) returns true iff succeeds
   removed ← false
   for each x in DOMAIN[Xi] do
      if no value y in DOMAIN[Xj] allows (x,y) to satisfy the constraint Xi ↔ Xj
         then delete x from DOMAIN[Xi];  removed ← true
   return removed
```

Limitations of Arc Consistency

After enforcing arc consistency:

- Can have one solution left
- Can have multiple solutions left
- Can have no solutions left (and not know it)

Arc consistency still runs inside a backtracking search!

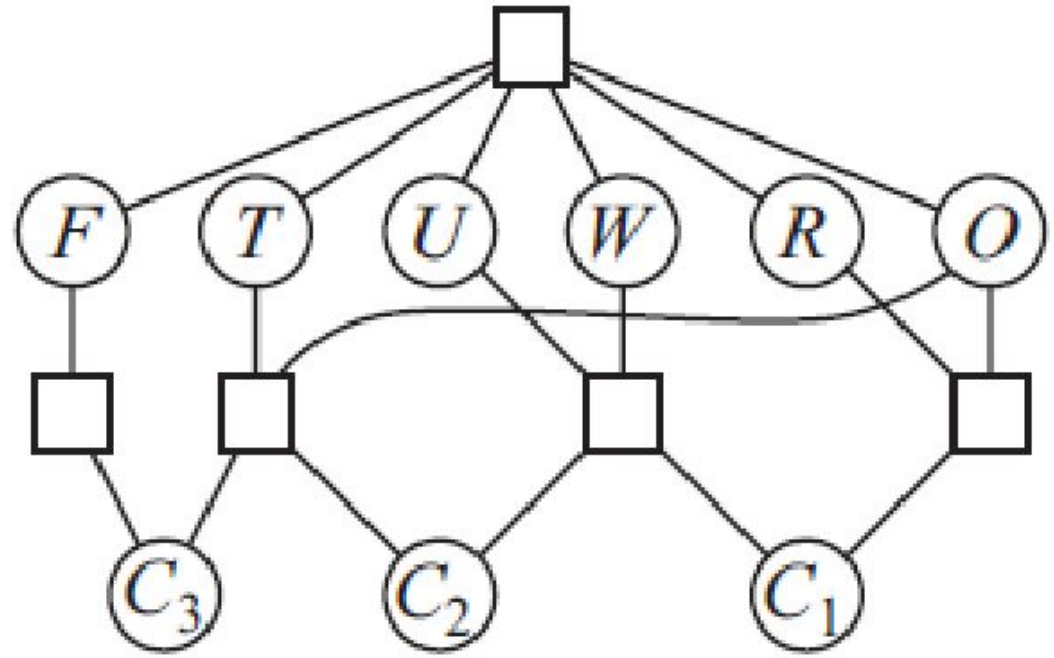


What went wrong here?

Cryptarithmetic

$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

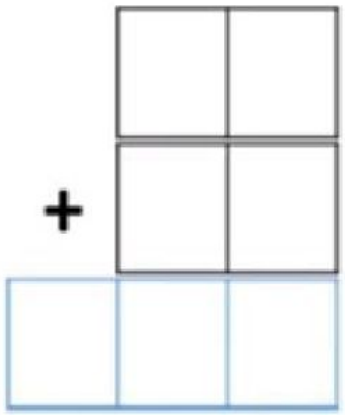
(a)



(b)

Figure 6.2 (a) A cryptarithmetic problem. Each letter stands for a distinct digit; the aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct, with the added restriction that no leading zeroes are allowed. (b) The constraint hypergraph for the cryptarithmetic problem, showing the *Alldiff* constraint (square box at the top) as well as the column addition constraints (four square boxes in the middle). The variables C_1, C_2, and C_3 represent the carry digits for the three columns.

Cryptarithmetic Problem



	T	O
+	G	O
O	U	T

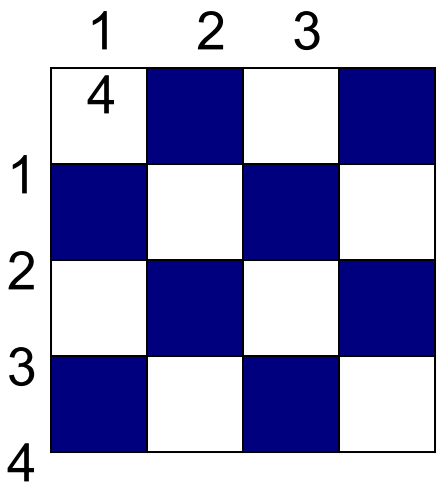
Character	Code
T	
O	
G	
U	

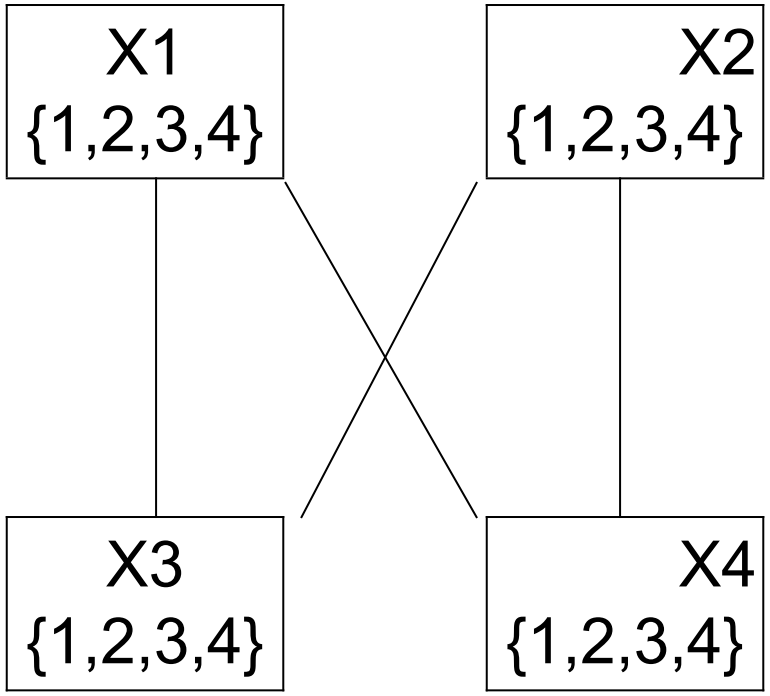
```
    S E N D
+   M O R E
-----------
M O N E Y
```

```
    □ □ □ □
+   □ □ □ □
-----------
□ □ □ □ □
```

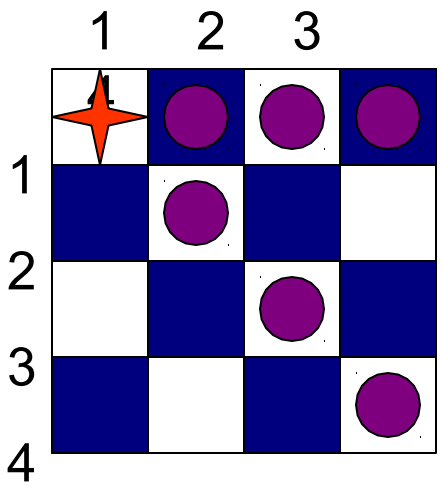
Character	Code
S	
E	
N	
D	
M	
O	
R	
Y	

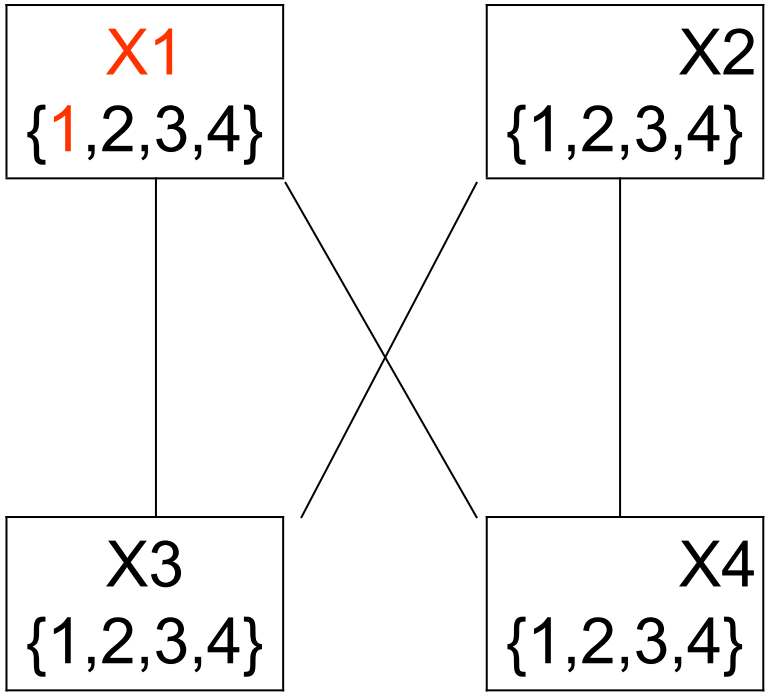
Example: 4-Queens Problem



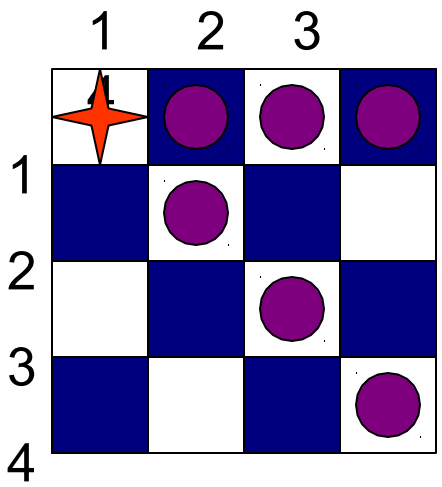


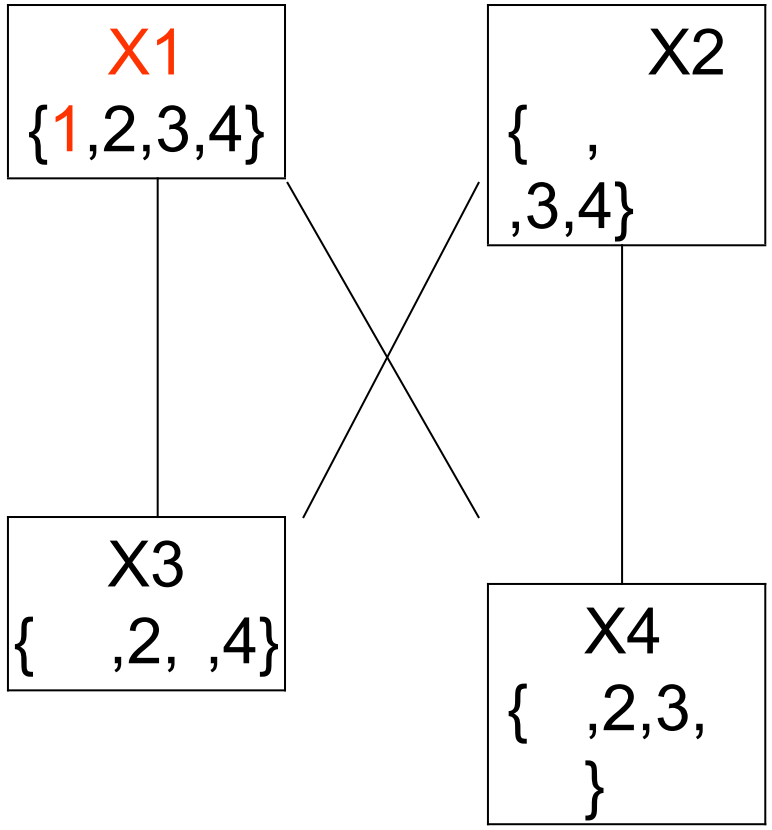
Example: 4-Queens Problem



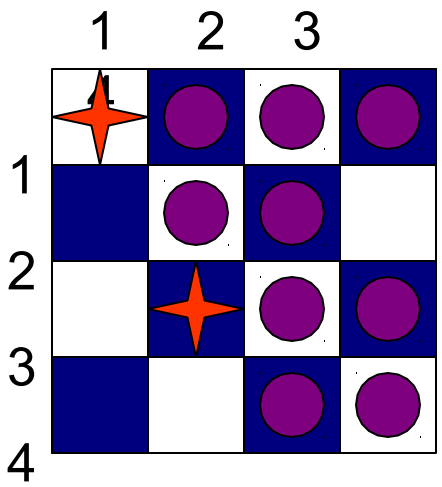


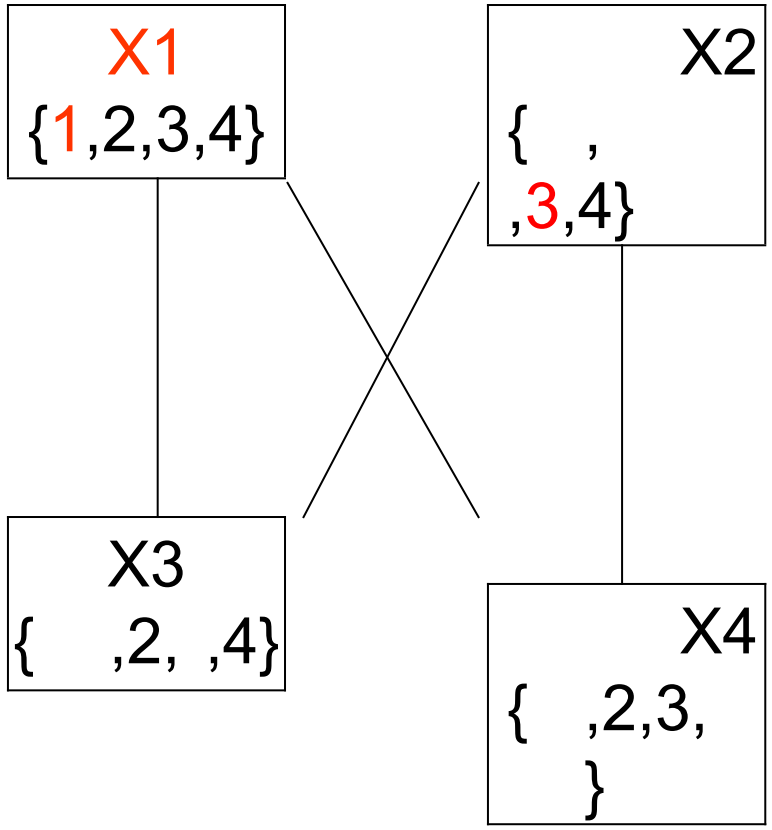
Example: 4-Queens Problem



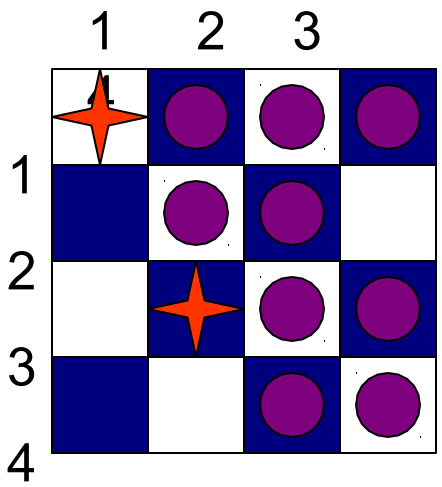


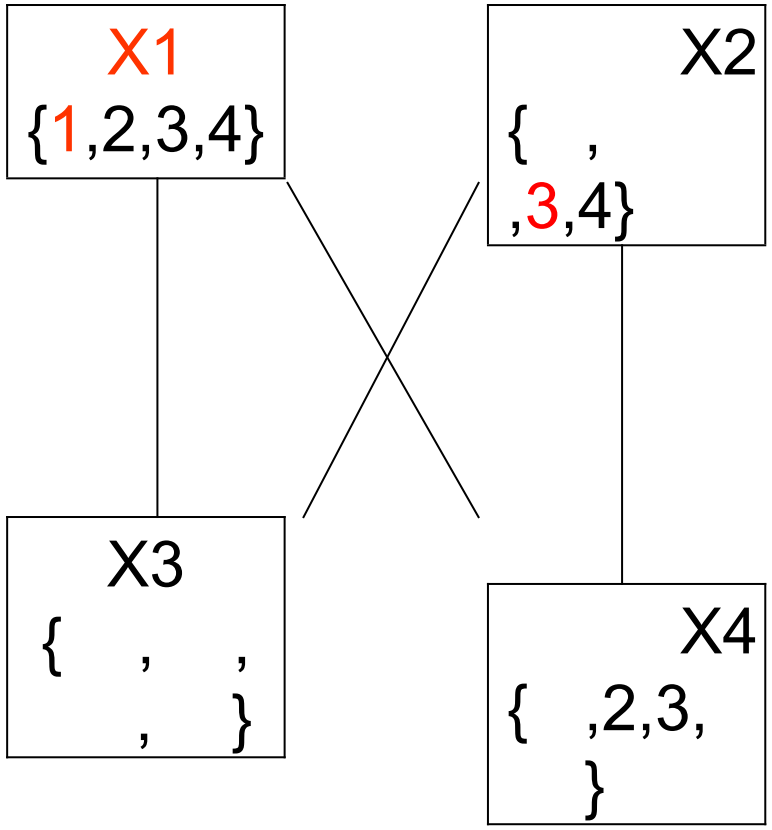
Example: 4-Queens Problem



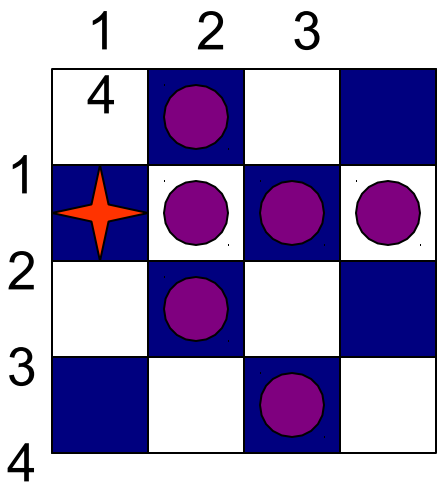


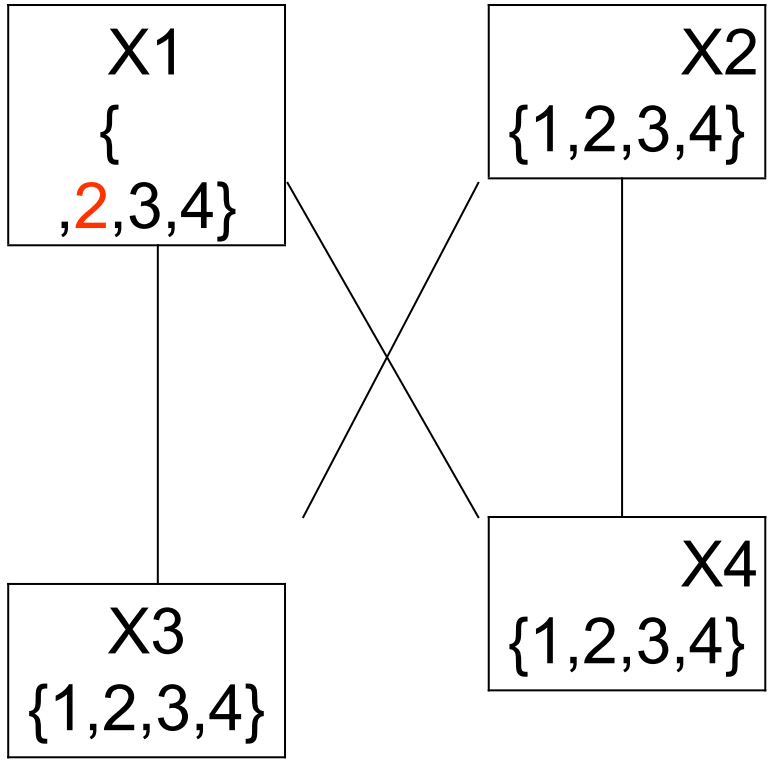
Example: 4-Queens Problem



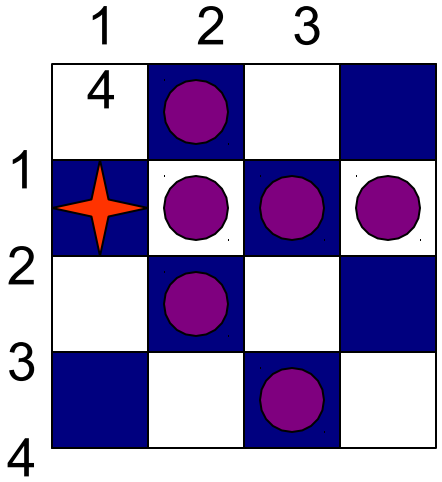


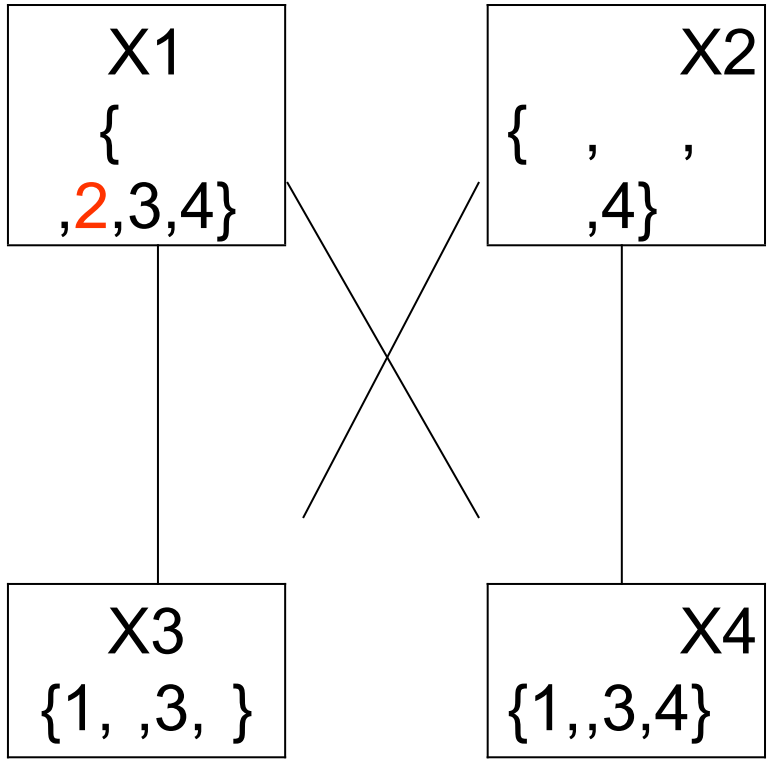
Example: 4-Queens Problem



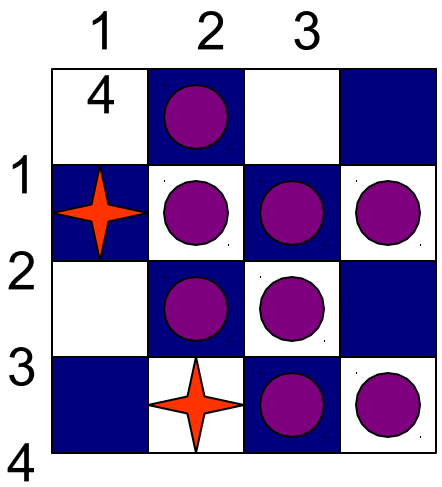


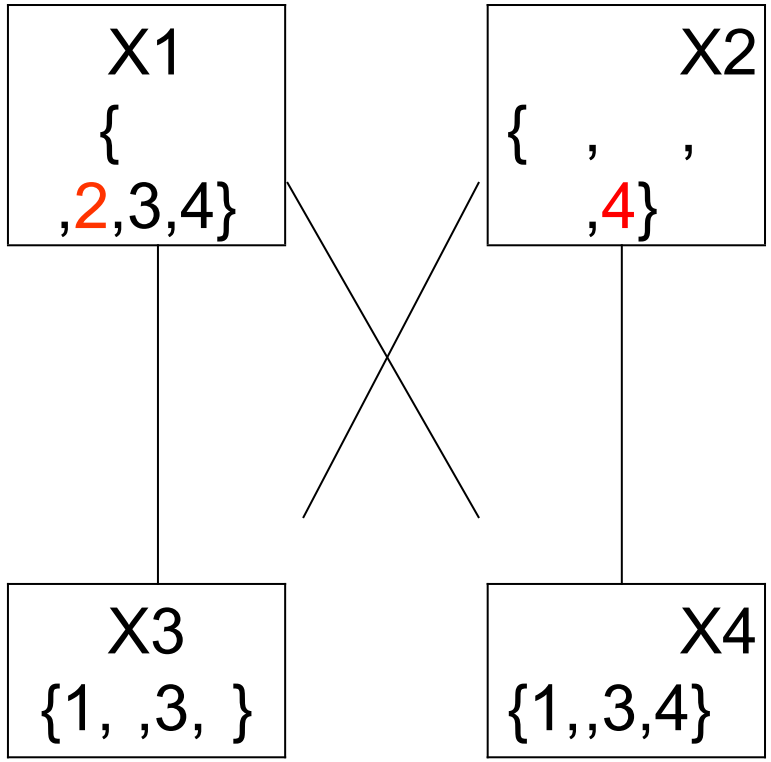
Example: 4-Queens Problem



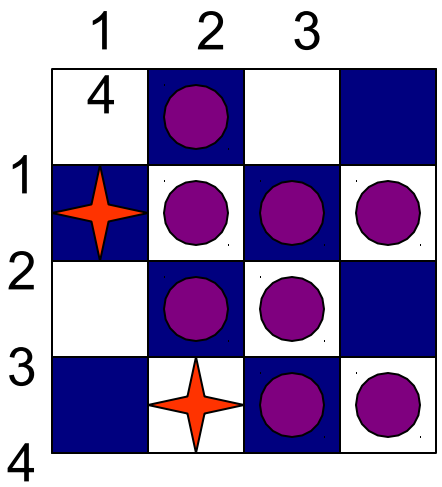


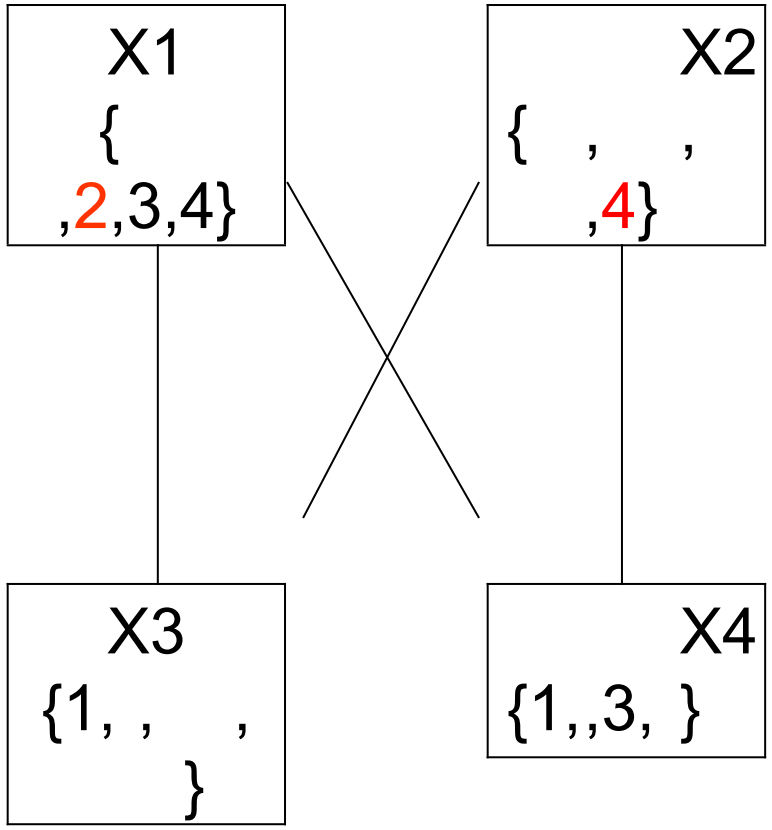
Example: 4-Queens Problem



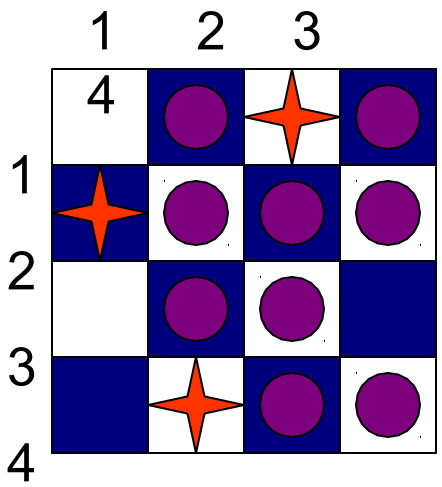


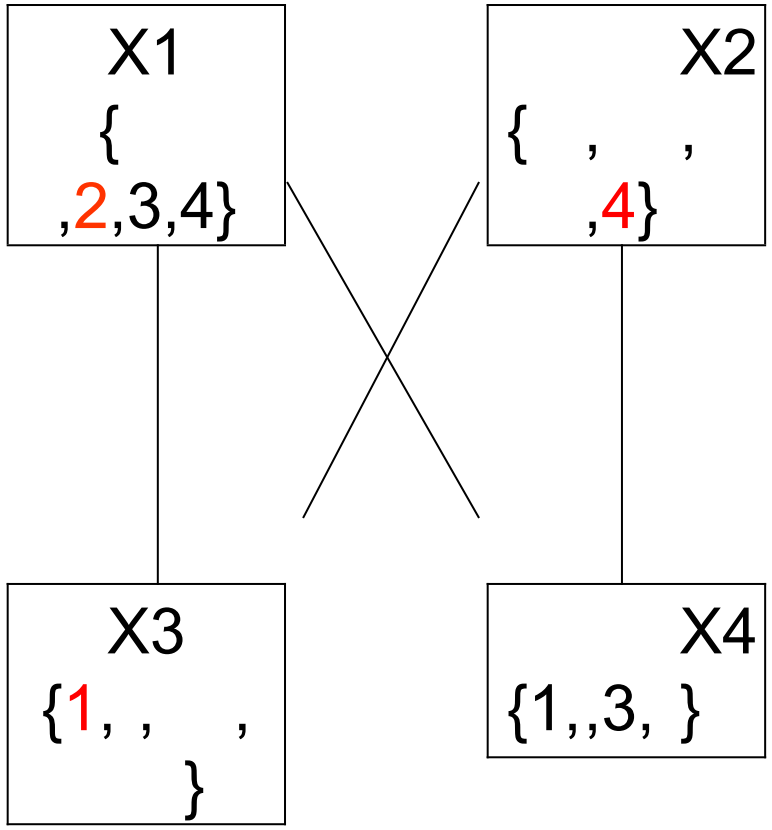
Example: 4-Queens Problem



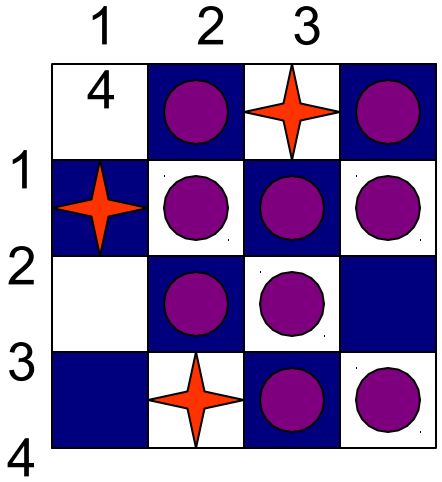


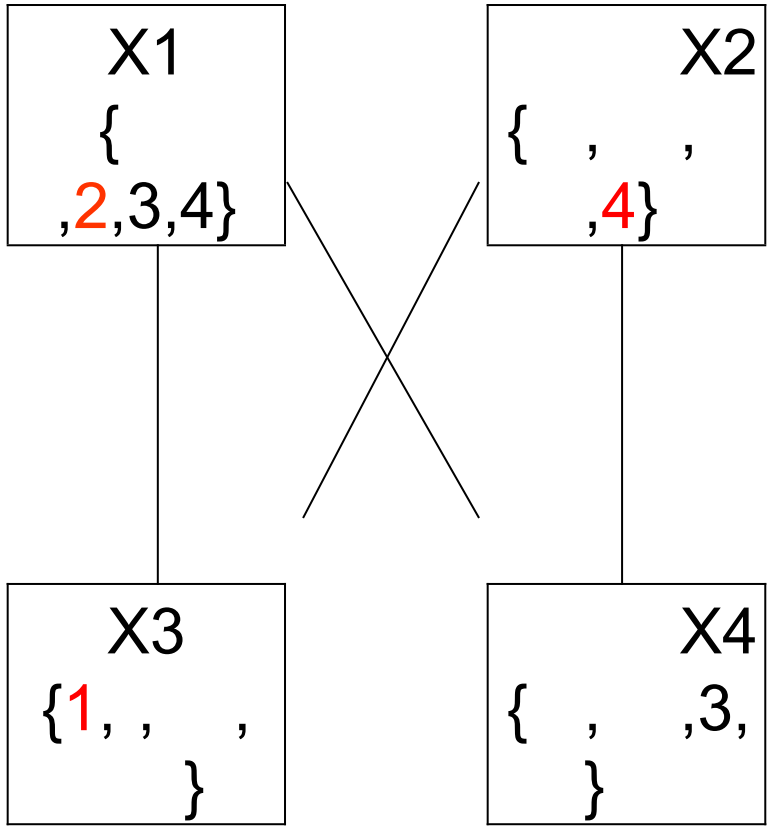
Example: 4-Queens Problem



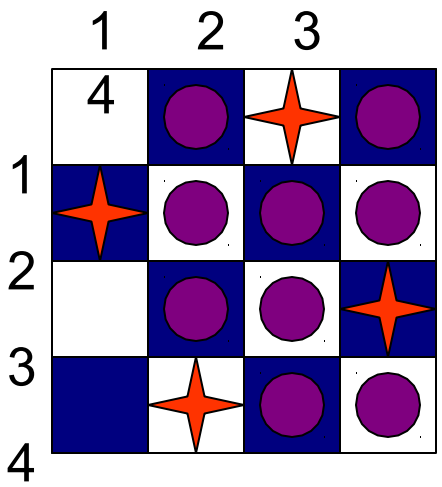


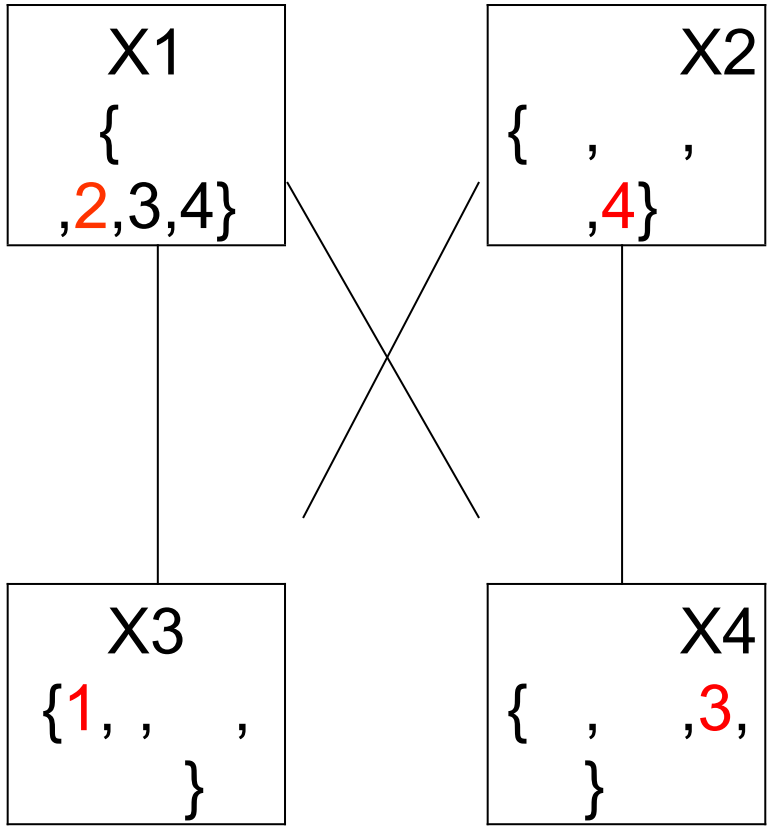
Example: 4-Queens Problem





Example: 4-Queens Problem





Intelligent Backtracking: Looking Backward

- Consider a fixed variable ordering Q, NSW, V , T, SA, WA, NT. Suppose we have generated the partial assignment {Q=red, NSW =green, V =blue, T =red}.
  When we try the next variable,
- SA, we see that every value violates a constraint(chronological backtracking)
- Recoloring Tasmania cannot possibly resolve the problem with South Australia.
- A more intelligent approach to backtracking is to backtrack to a variable that might fix the problem
- The set (in this case {Q=red ,NSW =green, V =blue, }), is called the conflict set for SA.
- In this case, back jumping would jump over Tasmania and try a new value for V .